

Lecture 24: Software Engineering

유준수 (Joon Soo Yoo, Ph.D.)

2026년 5월 12일

방산첨단보안연구실 (DAS Lab)

전북대학교 첨단방위산업학과

From Idea to Software System

요구사항부터 실제 구현까지

Software Engineering은 무엇일까?

많은 사람들은 프로그래밍과 Software Engineering을 같은 의미로 생각합니다.

하지만 실제로는 조금 다릅니다.

Programming

코드를 작성하여 기능을 구현하는 것

Software Engineering

- 무엇을 만들어야 하는지 고민
- 어떻게 구조를 나눌지 설계
- 유지보수 가능한 형태로 개발
- 여러 사람이 함께 개발 가능하게 구성

핵심

Software Engineering은
“코드 작성”보다 더 넓은 개념입니다.

왜 Software Engineering이 필요할까?

처음에는 작은 프로그램도 시간이 지나면 점점 커지게 됩니다.

초기 상태

- 코드 몇 줄
- 하나의 파일
- 혼자 개발 가능

하지만 프로젝트가 커지면:

문제 발생

- 코드 수정이 어려워짐
- 버그 증가
- 새로운 기능 추가 어려움
- 여러 사람이 함께 개발하기 어려움

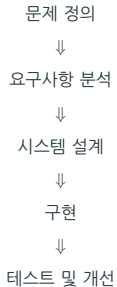
핵심

Software Engineering은
큰 프로그램을 관리 가능하게 만드는 방법입니다.

Software Engineering의 사고 과정

Software Engineering에서는 바로 코드를 작성하지 않습니다.

먼저 어떤 문제를 해결할 것인지 고민합니다.



핵심

Software Engineering은
“생각의 흐름”을 체계화하는 과정입니다.

먼저 해결하려는 문제는 무엇일까?

Software Engineering에서는
가장 먼저 “어떤 문제를 해결할 것인가”를 생각합니다.

현재 문제 상황

- 발표 평가를 수작업으로 진행
- 평균 계산이 번거로움
- 실시간 집계가 어려움
- 결과 저장 및 관리가 불편함

즉, 발표 평가를 더 효율적으로 관리할 수 있는 시스템이 필요했습니다.

핵심

Software Engineering은
현실의 문제를 해결하기 위해 시작됩니다.

왜 Requirement가 먼저일까?

많은 초보 개발자들은 바로 코딩을 시작하려고 합니다.

하지만 무엇을 만들어야 하는지 모른다면 문제가 발생합니다.

예를 들어

온라인 쇼핑몰을 만든다고 가정해봅시다.

먼저 생각해야 하는 것

- 회원가입이 필요한가?
- 결제 기능이 필요한가?
- 주문 내역 저장에 필요한가?
- 관리자 페이지가 필요한가?

핵심

코드를 작성하기 전에
“무엇이 필요한가”를 먼저 정의해야 합니다.

Requirement Analysis

무엇이 필요한가를 정의하기

Requirement란 무엇일까?

Requirement는 소프트웨어가 반드시 해야 하는 기능 또는 조건을 의미합니다.

즉,

“이 프로그램은 무엇을 해야 하는가?”

를 정리한 것입니다.

예시

온라인 쇼핑몰 시스템

Possible Requirements

- 회원가입 가능해야 함
- 상품 검색 가능해야 함
- 결제 가능해야 함
- 주문 내역 저장 가능해야 함

핵심

Requirement는
소프트웨어가 해야 하는 일을 정의합니다.

평가 시스템의 Requirement 생각해보기

발표 평가 시스템에도 여러 Requirement가 필요합니다.

현재 문제 상황

- 발표 평가를 빠르게 진행해야 함
- 여러 학생들의 점수를 집계해야 함
- 최종 결과를 저장해야 함

따라서 다음과 같은 기능들이 필요합니다.

현재 시스템의 Requirement

- 학생이 휴대폰으로 점수 입력 가능해야 함
- 점수가 저장되어야 함
- 평균이 자동 계산되어야 함
- 중복 제출이 방지되어야 함

Software Engineering Point

문제를 해결하기 위해 필요한 기능들을
먼저 정리하는 과정이 중요합니다.

Requirement에도 종류가 있을까?

Requirement는 크게 두 종류로 나눌 수 있습니다.

Functional Requirement

소프트웨어가 실제로 해야 하는 기능

예:

- 로그인 기능
- 점수 계산 기능
- 데이터 저장 기능

Non-functional Requirement

소프트웨어의 품질 또는 조건

예:

- 빠르게 동작해야 함
- 모바일에서 사용 가능해야 함
- 유지보수가 쉬워야 함

핵심

기능뿐 아니라 “어떤 품질을 가져야 하는가”도 중요합니다.

현재 프로젝트의 Requirement 분류

현재 평가 시스템의 Requirement를 분류해보면 다음과 같습니다.

Functional Requirements

- 학생 점수 입력
- 평균 계산
- 점수 저장
- 교수 점수 합산

Non-functional Requirements

- 모바일 접속 가능해야 함
- 중복 제출 방지 필요
- 결과 관리가 쉬워야 함
- 시스템 수정이 쉬워야 함

Software Engineering Point

Requirement를 정리하면
이후 시스템 설계가 쉬워집니다.

System Design

Requirement를 실제 시스템으로 바꾸기

Requirement만 있으면 충분할까?

Requirement를 정의했다고 해서
바로 프로그램이 만들어지는 것은 아닙니다.

이제는 다음 질문이 필요합니다.

“이 기능들을 어떻게 구성할 것인가?”

예를 들어:

- 어디에서 점수를 입력할까?
- 어디에서 데이터를 저장할까?
- 평균 계산은 누가 담당할까?
- 교수는 어디에서 결과를 볼까?

핵심

Requirement 이후에는
시스템 구조(System Design)를 고민해야 합니다.

현재 시스템에는 어떤 역할이 필요할까?

발표 평가 시스템을 다시 생각해봅시다.

학생은 점수를 입력해야 하고,
교수는 결과를 확인해야 합니다.

또한 시스템은 점수를 저장하고 계산해야 합니다.

필요한 역할들

- 학생 → 점수 입력
- 교수 → 결과 확인
- 서버 → 요청 처리
- 계산 로직 → 평균 계산
- 저장 공간 → 데이터 저장

핵심

하나의 프로그램 안에서도
여러 역할(Role)이 존재합니다.

왜 역할을 나누어야 할까?

처음에는 하나의 파일에 모든 코드를 작성할 수도 있습니다.

하지만 프로젝트가 커질수록 문제가 발생합니다.

모든 기능이 하나의 파일에 있다면?

- 코드가 너무 길어짐
- 어디를 수정해야 하는지 어려움
- 오류 찾기가 힘들어짐
- 다른 사람과 협업이 어려움

따라서 역할별로 구조를 나누기 시작합니다.

Software Engineering Point

큰 프로그램일수록 역할 분리가 중요합니다.

현재 프로젝트의 역할 분리

현재 프로젝트에서는 역할을 다음과 같이 나누었습니다.

현재 구조

- Student Page → 학생 점수 입력
- Flask Server → 요청 처리
- Calculator → 평균 계산
- Storage → 데이터 저장
- Professor Dashboard → 결과 확인

각 역할은 서로 다른 책임을 가집니다.

예를 들어:

예시

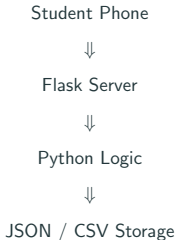
Calculator는 계산만 담당하고,
Storage는 저장만 담당합니다.

핵심 개념

Separation of Concerns

현재 시스템의 전체 흐름

현재 시스템은 아래와 같은 흐름으로 동작합니다.



학생은 휴대폰으로 점수를 입력하고,
서버는 데이터를 처리한 뒤 저장합니다.

Software Engineering Point

Software System은 여러 구성 요소가 함께 동작하는 구조입니다.

현재 프로젝트 구조

실제 프로젝트는 아래와 같은 구조로 구성했습니다.

프로젝트 구조

se_evaluation_system/

- |— app.py
- |— services/
- |— templates/
- |— data/

각 폴더의 역할

- app.py → 전체 흐름 관리
- services/ → 계산 및 저장 로직
- templates/ → 사용자 화면
- data/ → 실제 데이터 저장

핵심

좋은 구조는 유지보수와 기능 확장을 쉽게 만듭니다.

왜 이런 구조를 선택했을까?

현재는 작은 프로젝트처럼 보일 수 있습니다.

하지만 이후 더 많은 기능이 추가될 수 있습니다.

추가 가능한 기능

- 로그인 기능
- QR 코드 접속
- 실시간 통계
- 클라우드 서버 배포
- 데이터베이스 연동

초기에 구조를 잘 나누어두면
이후 기능 추가가 훨씬 쉬워집니다.

Software Engineering Point

Software Engineering에서는
미래의 변화와 확장성까지 고려합니다.

Incremental Development

작은 기능부터 점진적으로 확장하기

처음부터 완벽한 시스템을 만들 수 있을까?

많은 초보 개발자들은
처음부터 모든 기능을 한 번에 만들려고 합니다.

하지만 실제로는 매우 어렵습니다.

문제점

- 오류 발생 가능성 증가
- 어디가 문제인지 찾기 어려움
- 개발 속도 저하
- 구조가 복잡해짐

따라서 실제 개발에서는 작은 기능부터 차례대로 구현합니다.

핵심

Software Development는
작은 단계들의 반복 과정입니다.

현재 프로젝트는 어떻게 시작되었을까?

처음부터 모든 기능을 만들지 않았습니다.

가장 먼저 아주 작은 기능부터 시작했습니다.

첫 번째 목표

Flask 서버가 정상적으로 실행되는지 확인

즉,



구조가 먼저 동작하는지 테스트했습니다.

Software Engineering Point

큰 시스템도 작은 기능부터 하나씩 검증하며 개발합니다.

학생 입력 기능은 어디에 추가했을까?

기본 Flask 서버가 동작한 이후,
학생들이 실제로 점수를 입력할 수 있는 기능이 필요했습니다.

따라서 프로젝트 내부에 학생 입력 화면을 추가했습니다.

추가된 구성 요소

- templates/student.html
- 학생 이름 선택 기능
- 점수 입력 기능
- 제출 버튼

즉, 시스템을 사용자와 상호작용 가능한 형태로 발전 시켰습니다.

데이터 저장 기능은 어디에 추가했을까?

학생 점수가 프로그램 종료 후 사라지지 않도록
데이터 저장 기능이 필요했습니다.

따라서 저장 기능을 별도의 모듈로 분리했습니다.

추가된 구성 요소

- services/score_service.py
- data/scores.json
- 점수 저장 기능
- 점수 불러오기 기능

저장 기능을 따로 분리하면
나중에 Database로 변경하기도 쉬워집니다.

핵심 개념

Data Persistence + Modularity

Software Engineering Concepts

프로젝트 안에 숨어있는 설계 개념들

왜 저장 기능을 따로 분리했을까?

이전 슬라이드에서 저장 기능을 별도의 위치에 추가했습니다.

```
services/score_service.py
```

그 이유는 단순히 코드를 정리하기 위해서만은 아닙니다.

생각해볼 문제

현재는 JSON 파일을 사용하고 있지만,
나중에는 Database를 사용할 수도 있습니다.

예:

- SQLite
- MySQL
- PostgreSQL

핵심

저장 기능을 따로 분리하면
나중에 구조를 변경하기 쉬워집니다.

Data Persistence란 무엇일까?

소프트웨어에서는 프로그램이 종료되어도
데이터가 유지되어야 하는 경우가 많습니다.

예를 들어:

- 게임 저장 데이터
- 로그인 정보
- 게시물
- 발표 평가 결과

이처럼 데이터를 계속 유지하는 개념을
Data Persistence라고 부릅니다.

현재 프로젝트에서는

- scores.json
- final_scores.csv

핵심

Software System에서는
데이터 저장과 관리가 매우 중요합니다.

Modularity란 무엇일까?

프로그램을 하나의 거대한 파일로 만드는 대신,
기능별로 나누는 방식을 Modularity라고 합니다.

현재 프로젝트의 예시

- calculator.py → 계산 기능
- score_service.py → 저장 기능
- student.html → 입력 화면

각 파일은 서로 다른 역할을 담당합니다.

왜 중요할까?

- 수정하기 쉬움
- 오류 찾기 쉬움
- 기능 추가 쉬움
- 여러 사람이 함께 개발 가능

핵심

큰 프로그램일수록
기능을 나누는 구조가 중요해집니다.

현재 프로젝트에서의 Modularity

현재 프로젝트 구조를 다시 살펴보면:

프로젝트 구조

se_evaluation_system/

```
|— app.py
|— services/
|— templates/
|— data/
```

각 폴더는 서로 다른 역할을 담당합니다.

- services/ → 계산 및 저장
- templates/ → 사용자 화면
- data/ → 실제 데이터

즉, 시스템 전체를 작은 기능 단위로 나누어 관리하고 있습니다.

Software Engineering Point

좋은 구조는 유지보수와 확장을 쉽게 만듭니다.

평균 계산 기능은 어디에 추가했을까?

발표 평가 시스템에서는
자동 평균 계산 기능이 필요했습니다.

계산 기능 역시 별도의 모듈로 분리했습니다.

추가된 구성 요소

- services/calculator.py
- 평균 계산 기능
- 절단 평균(Trimmed Mean) 계산

계산 기능을 따로 분리하면
이후 계산 규칙을 수정하기 쉬워집니다.

예:

- 일반 평균
- 절단 평균
- 가중 평균

Software Engineering Point

Business Logic을 별도 모듈로 분리했습니다.

Business Logic이란 무엇일까?

Business Logic은
소프트웨어가 따라야 하는 “운영 규칙” 입니다.

즉,

“이 시스템은 어떤 규칙으로 동작해야 하는가?”

를 코드로 표현한 것입니다.

예시

온라인 쇼핑몰 시스템

Possible Business Logic

- 5만원 이상 구매 시 무료 배송
- VIP 회원은 10% 할인
- 쿠폰은 한 번만 사용 가능

핵심

Business Logic은
현실 세계의 규칙을 코드로 구현한 것입니다.

현재 프로젝트의 Business Logic

현재 발표 평가 시스템에도
여러 운영 규칙들이 존재합니다.

현재 시스템의 규칙들

- 최고점 제거
- 최저점 제거
- 절단 평균 계산
- 학생 점수 50%
- 교수 점수 50%
- 중복 제출 방지

이러한 규칙들을
`services/calculator.py` 안에서 처리했습니다.

즉, 계산 모듈이 단순 계산만 하는 것이 아니라
평가 시스템의 운영 규칙도 함께 관리하고 있습니다.

Software Engineering Point

Business Logic은 별도의 모듈로 분리하는 경우가 많습니다.

프로젝트 구조는 어떻게 발전했을까?

기능이 점점 추가되면서
프로젝트 구조도 함께 확장되었습니다.

현재 프로젝트 구조

se_evaluation_system/

```
├── app.py
├── services/
│   ├── calculator.py
│   └── score_service.py
├── templates/
│   └── student.html
└── data/
    └── scores.json
```

각 기능이 서로 다른 위치에 분리되면서
프로젝트 구조가 점점 체계적으로 변화했습니다.

핵심

Software System은 기능이 증가할수록
구조적인 설계가 더욱 중요해집니다.

현재 시스템은 계속 확장 가능하다

현재 시스템은 아직도 계속 발전할 수 있습니다.

예를 들어 다음과 같은 기능들을 추가할 수 있습니다.

추가 가능한 기능

- 로그인 기능
- QR 코드 접속
- 실시간 통계
- SQLite / Database 연동
- 클라우드 서버 배포

초기에 구조를 잘 나누어두면
새로운 기능 추가가 훨씬 쉬워집니다.

Software Engineering Point

Software는 한 번 만들고 끝나는 것이 아니라
지속적으로 수정되고 확장됩니다.

Testing and Validation

구현 이후에는 무엇이 필요할까?

구현했다고 끝일까?

프로그램이 실행된다고 해서
항상 올바르게 동작하는 것은 아닙니다.

예를 들어:

- 잘못된 점수 입력
- 중복 제출
- 평균 계산 오류
- 저장 실패

같은 문제들이 발생할 수 있습니다.

따라서 Software Engineering에서는
구현 이후 Testing 과정이 매우 중요합니다.

핵심

좋은 소프트웨어는
“동작하는 것” 뿐 아니라 “올바르게 동작”해야 합니다.

현재 프로젝트에서는 무엇을 테스트했을까?

현재 발표 평가 시스템에서도
여러 기능들을 테스트했습니다.

현재 테스트한 기능

- 학생 점수 입력 가능 여부
- 평균 계산 결과 확인
- 최고점 / 최저점 제거 확인
- 중복 제출 방지 확인
- JSON / CSV 저장 확인

특히 계산 기능은 작은 오류 하나만 있어도
최종 성적 결과가 달라질 수 있습니다.

Software Engineering Point

Testing은 시스템의 신뢰성을 높이는 과정입니다.

Edge Case란 무엇일까?

소프트웨어에서는
특수한 입력 상황도 고려해야 합니다.

이러한 특수 상황을 Edge Case라고 부릅니다.

현재 프로젝트의 Edge Case

- 점수가 하나도 없는 경우
- 동일 학생이 여러 번 제출하는 경우
- 최고점과 최저점이 같은 경우
- 잘못된 점수를 입력하는 경우

실제 Software Engineering에서는
이러한 예외 상황 처리도 매우 중요합니다.

핵심

좋은 소프트웨어는
정상 상황뿐 아니라 예외 상황도 고려합니다.

Refactoring and Evolution

소프트웨어는 계속 변화한다

처음부터 완벽한 구조였을까?

실제 개발에서는 처음부터 완벽한 구조를 만들기 어렵습니다.

처음에는 단순하게 시작하고,
기능이 늘어나면서 구조를 개선해나갑니다.

현재 프로젝트의 변화

- 단순 Flask 서버
- 학생 입력 기능 추가
- 데이터 저장 기능 추가
- 계산 기능 분리
- 구조 및 역할 분리

즉, 프로젝트 구조도 점점 발전했습니다.

핵심

Software Development는
계속 수정되고 발전하는 과정입니다.

Refactoring이란 무엇일까?

Refactoring은

기능은 유지하면서 내부 구조를 개선하는 과정입니다.

예를 들어:

초기 상태

모든 기능이 app.py 하나에 존재

Refactoring 이후

- calculator.py
- score_service.py
- templates/
- data/

즉, 기능은 같지만 구조가 더 좋아졌습니다.

Software Engineering Point

Refactoring은 유지보수성과 확장성을 높이는 중요한 과정입니다.

프로젝트가 커지면 Refactoring은 계속 필요할까?

실제 Software Development에서는
프로젝트가 커질수록 Refactoring이 계속 발생합니다.

왜냐하면 처음에는 작은 프로젝트였지만,
시간이 지나면서 기능이 계속 추가되기 때문입니다.

프로젝트의 변화

- 처음 → 단순 Flask 서버
- 이후 → 학생 입력 기능 추가
- 이후 → 데이터 저장 기능 추가
- 이후 → 평균 계산 및 검증 기능 추가

기능이 증가할수록 구조도 함께 변화해야 합니다.

핵심

Software는 계속 변화하기 때문에
Refactoring도 반복적으로 발생합니다.

왜 처음부터 완벽하게 나누지 않을까?

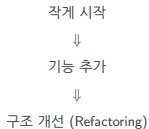
그렇다면 처음부터 완벽한 구조로 만들면 되지 않을까요?

하지만 실제로는 매우 어렵습니다.

초기 개발 단계

- 어떤 기능이 추가될지 모름
- 프로젝트 규모를 정확히 예측하기 어려움
- 너무 복잡하게 시작하면 개발 속도가 느려짐

따라서 실제 개발에서는:



과정을 반복하는 경우가 많습니다.

Software Engineering Point

좋은 구조는 한 번에 완성되는 것이 아니라 점진적으로 발전합니다.

Summary

Software Engineering 개념들

Software Engineering Concepts Summary

SE Concept	Description
Problem Definition	어떤 문제를 해결할 것인지 정의하는 과정
Requirement Analysis	필요한 기능과 조건을 정리하는 과정
System Design	전체 시스템 구조를 설계하는 과정
Modularity	기능을 작은 단위로 나누어 관리하는 방법
Data Persistence	프로그램 종료 후에도 데이터를 유지하는 개념
Business Logic	시스템이 따라야 하는 운영 규칙
Incremental Development	작은 기능부터 점진적으로 확장하는 개발 방식
Testing	프로그램이 올바르게 동작하는지 확인하는 과정
Refactoring	기능은 유지하면서 내부 구조를 개선하는 과정
Software Evolution	소프트웨어가 지속적으로 수정되고 발전하는 과정

핵심

Software Engineering은 단순 코딩이 아니라 “구조적으로 문제를 해결하는 과정” 입니다.

Any Questions?